

ユニット 4・第5-6週

プロセスとメモリ

📅 第5-6週・日程は未定 </> 演習 TD3（解答 + 生成AIチャレンジ） ~ アニメーション 7 本

演習 TD3・解答付き

バッファ、`fork`、`exec`、`wait`

C ライブラリがシステムコールの上で何をしているか、プロセスが自分を複製するとき何がコピーされるか、そしてプログラムが別のプログラムを起動し、その終わり方を知る方法。

☑ 実習レポート

この実習は、授業のテンプレートを使った短い LaTeX レポートで提出します。各演習の説明、要点となるコード、端末で実行したことを書きます。LaTeX の作成には生成 AI を使えます。利用を申告し、すべての結果を確認してください。テンプレート、例、Prism のガイド →

演習問題

- | | | | |
|---|-------------------------------------|---|---|
| 1 | システムコールかライブラリ関数か？ | 4 | <code>launch</code> ：リダイレクトして別のプログラムを実行する |
| 2 | <code>stdio</code> の 3 つのバッファリングモード | 5 | 探究：仮想メモリとプロセスの一生（新規） |
| 3 | <code>fork</code> と複製されたバッファ | | |

TD3 ・ 1

● 解答済み

システムコールかライブラリ関数か？

課題。次を分類してください：`printf`、`fopen`、`fclose`、`fread`、`open`、`close`、`lseek`、`rewind`、`write`、`exit`、`_exit`。プロトタイプとマニュアルのセクションを手がかりにしましょう。

解答

システムコール（セクション 2）

`open`、`close`、`lseek`、`write`、`_exit`

C ライブラリ関数（セクション 3）

`printf`、`fopen`、`fclose`、`fread`、`rewind`、`exit`

手がかりは型にあります：システムコールはディスクリプタ（`int fd`）を、`stdio` の関数は `FILE *`（ディスクリプタとバッファを持つライブラリの構造体）を使います。 `exit` はそれらのバッファをフラッシュし、 `atexit` の関数を実行してから、プロセスを即座に終了する `_exit` を呼びます。

TD3 ・ 2

● 解答済み・実際の出力

stdio の 3 つのバッファリングモード

課題。 `buffer.c` は「Hello」と書き、1秒眠ってから「World!」と書きます。プリプロセッサのマクロで、各文字列ごとに `fprintf` か `write`、`stdout` か `stderr`、最後に `exit` か `_exit` を選びます。各バージョンを端末とパイプ経由（`2>&1 | cat -u`）で実行し、3つのバッファリングモードを見分けてみましょう。

解答

C tds/td3/buffer.c

```
/*
 * TD3 - exercise 2: the three buffering modes of the C library.
 * One source, several behaviours chosen at compile time (see the Makefile
 * variants):
 *   STRING1, STRING2           the two strings (default "Hello " and
 *                               "World!\n")
 *   USE_FPRINTF1/2             write string 1/2 with fprintf (buffered) instead
 *                               of write (not buffered)
 *   USE_STDERR1/2              write string 1/2 on stderr instead of stdout
 *   USE__EXIT                   end with _exit() (no buffer flush) instead of
 *                               exit()
 * Run each variant twice: ./buffer_N (terminal) and ./buffer_N 2>&1 |
 * cat -u (pipe)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

#ifndef STRING1
#define STRING1 "Hello "
```

```
#endif

#ifndef STRING2
#define STRING2 "World!\n"
#endif

#ifdef USE_STDERR1
#define STREAM1 stderr
#define FD1 STDERR_FILENO
#else
#define STREAM1 stdout
#define FD1 STDOUT_FILENO
#endif

#ifdef USE_STDERR2
#define STREAM2 stderr
#define FD2 STDERR_FILENO
#else
#define STREAM2 stdout
#define FD2 STDOUT_FILENO
#endif

int main(void)
{
#ifdef USE_FPRINTF1
    fputs(STRING1, STREAM1);           /* goes to the
    stdio buffer */
#else
    exit_if(write(FD1, STRING1, strlen(STRING1)) == -1, "write"); /*
    straight to the kernel */
#endif

    sleep(1);

#ifdef USE_FPRINTF2
    fputs(STRING2, STREAM2);
#else
    exit_if(write(FD2, STRING2, strlen(STRING2)) == -1, "write");
#endif

#ifdef USE__EXIT
    _exit(EXIT_SUCCESS);               /* system call: the stdio buffers are NOT
    flushed */
#else
    exit(EXIT_SUCCESS);               /* library function: flushes the stdio
    buffers, then _exit */
}
```

```
#endif
}
```

バージョン (MAKEFILE)	端末	パイプ	理由
buffer_1 : fprintf(stdout) + write	World! · Hello	World! · Hello	「Hello」には \n がない：行バッファリングでもフラッシュされず、exit で初めて出る。
buffer_2 : write + fprintf(stderr)	Hello World!	Hello World!	stderr はバッファされない：fprintf のたびに即座に write する。
buffer_3 : fprintf(stdout) + fprintf(stderr)	World! · Hello	World! · Hello	エラー出力はすぐ出て、標準出力は待つ。
buffer_4 : 3 と同じで "Hello\n"	Hello · World!	World! · Hello	端末では stdout は行バッファリング (\n でフラッシュ)、パイプでは完全バッファリング (4 KiB) なので exit まで待つ。
buffer_5 : fprintf(stdout) + _exit	World!	World!	_exit はバッファをフラッシュしない：「Hello」は失われる。

実際の出力

```
$ ./buffer_4          # en una terminal
Hello
World!
```

実際の出力

```
$ ./buffer_4 2>&1 | cat -u
World!
Hello
```

実際の出力

```
$ ./buffer_5 2>&1 | cat -u
World!
```

3つのモード：バッファなし（`stderr`）、行バッファ（端末への `stdout`）、完全バッファ（ファイルやパイプへの `stdout`）。`write` にはプログラムから見えるバッファはありません：渡したものはすでにカーネルの手元にあります。

アニメーション A4.6・stdio のバッファ →

TD3 ・ 3

● 解答済み・テスト済み

fork と複製されたバッファ

課題。プログラムが `printf` で自分の pid を改行なしで表示し、`fork` で複製します。子は自分の pid と親の pid を表示します。親は `fork` の後に何も表示してはいけません。何が起きますか？解決策を 3 つ挙げてください。

解答

```
C tds/td3/fork_pid.c

/*
 * TD3 - exercise 3: print the pid WITHOUT a newline, fork, and let only the
 * child print.
 * SOLUTION=0 the bug: "My pid is ..." appears twice (the unflushed stdio
 * buffer is copied by fork)
 * SOLUTION=1 fflush(stdout) before fork()
 * SOLUTION=2 no stdio buffer for stdout: setvbuf(stdout, NULL, _IONBF, 0)
 * SOLUTION=3 the parent leaves with _exit(): its copy of the buffer is
 * never flushed,
 * the child's copy is printed together with the child's line
 * Make builds the four variants: fork_pid_0 ... fork_pid_3.
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#ifndef SOLUTION
#define SOLUTION 1
#endif
```

```
int main(void)
{
    #if SOLUTION == 2
        exit_if(setvbuf(stdout, NULL, _IONBF, 0) != 0, "setvbuf");
    #endif

    printf("My pid is %d / ", getpid());    /* no '\n': stays in the buffer
    */
    #if SOLUTION == 1
        fflush(stdout);
    #endif

    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                          /* child */
        printf("child %d, parent %d\n", getpid(), getppid());
        exit(EXIT_SUCCESS);                  /* exit flushes the child's
    buffer */
    }

    /* Parent: prints nothing after fork, but must wait for its child. */
    int status;
    exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
    #if SOLUTION == 3
        _exit(EXIT_SUCCESS);                  /* do not flush the parent's
    copy */
    #else
        return EXIT_SUCCESS;                  /* returning from main =
    exit(): flushes */
    #endif
}
```

実際の出力

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

実際の出力

```
$ ./fork_pid_1 | cat
My pid is 356458 / child 356460, parent 356458
```

実際の出力

```
$ ./fork_pid_3 | cat
My pid is 356482 / child 356484, parent 356482
```

問題点。「My pid is ... /」はプロセスのメモリにある `stdout` のバッファに残っています。`fork` はバッファを含むメモリ全体をコピーするので、親と子が同じ未出力のテキストを持ち、どちらも終了時にフラッシュします。だから親は「何も表示しない」のに 2 回表示されるのです。**解決策：**(1) `fork` の前に `fflush(stdout)`、(2) `setvbuf(..., _IONBF, 0)` でバッファをなくす、(3) 親をコピーをフラッシュしない `_exit` で終わらせる：テキストは子から 1 回だけ出ます。最も明快なのは 1 で、一般的な規則は `fork` の前にバッファをフラッシュすることです。

TD3 ・ 4

● 解答済み・テスト済み

launch：リダイレクトして別のプログラムを実行する

課題。 `./launch output command arg1 arg2...` は `command arg1 arg2... > output` のように振る舞う必要があります：コマンドラインを `output` に書き、標準出力を `dup2` でリダイレクトしてから `execvp` でコマンドを実行する子プロセスを作り、親がその終わり方を報告します (`wait(2)`)。セグメンテーション違反を起こすプログラムで試しましょう。

解答

C tds/td3/launch.c

```
/*
 * TD3 - exercise 4: ./launch output command [args...] behaves like command
 * args... > output
 * 1. writes the command line into `output`
 * 2. forks; the child redirects its stdout to `output` with dup2 and runs
 * the command (execvp)
 * 3. the parent waits and reports how the command ended (exit status or
 * signal)
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
```

```
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    if (argc < 3) {
        fprintf(stderr, "usage: %s output command [args...]\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* O_TRUNC: like "> output", previous content is discarded. 0644 = rw-r--
r-- */
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, argv[1]);

    for (int i = 2; i < argc; i++) {
        exit_if(write_all(fd, argv[i], strlen(argv[i])) == -1, "write");
        exit_if(write_all(fd, i < argc - 1 ? " " : "\n", 1) == -1, "write");
    }

    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {
        /* child */
        exit_if(dup2(fd, STDOUT_FILENO) == -1, "dup2");
        exit_if(close(fd) == -1, "close"); /* 1 now points to the
file */
        execvp(argv[2], argv + 2); /* only returns on
error */
        perror(argv[2]);
        _exit(127); /* like the shell:
command not found */
    }

    exit_if(close(fd) == -1, "close"); /* the parent does not
need it */

    int status;
    exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
    if (WIFEXITED(status)) {
        fprintf(stderr, "launch: %s exited with status %d\n", argv[2],
WEXITSTATUS(status));
        return WEXITSTATUS(status);
    }
}
```

```

    if (WIFSIGNALED(status)) {
        fprintf(stderr, "launch: %s killed by signal %d (%s)%s\n", argv[2],
WTERMSIG(status),
            signame(WTERMSIG(status)), WCOREDUMP(status) ? ", core
dumped" : "");
        return 128 + WTERMSIG(status);          /* shell convention */
    }
    return EXIT_FAILURE;
}

```

C tds/td3/segfault.c

```

/*
 * TD3 - exercise 4: a program that dies with SIGSEGV, to test the error path
of launch.
 * Address 0 is never mapped: the MMU raises a page fault, the kernel finds
no mapping and
 * sends SIGSEGV to the process.
 */
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

static void gen_segfault(void)
{
    volatile int *p = NULL;    /* volatile: the compiler must really do the
store */
    *p = 42;
}

int main(void)
{
    printf("about to write at address 0...\n");
    fflush(stdout);
    gen_segfault();
    return EXIT_SUCCESS;      /* never reached */
}

```

実際の出力

```
$ ./launch out.txt ls -l demo; cat out.txt
launch: ls exited with status 0
ls -l demo
a.txt
b.txt
```

実際の出力

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

実際の出力

```
$ ./launch out.txt nocommand
nocommand: No such file or directory
launch: nocommand exited with status 127
[exit status 127]
```

- `execvp` はディスクリプタを残したままプロセスのプログラムを置き換えます：だからリダイレクトは子の中で、`fork` と `exec` の間に行うのです。シェルはこうして動いています。
- `execvp` が戻るのは失敗したときだけです：次の行は必ずエラー処理です（127 = 「見つからない」、シェルの慣例）。親からコピーしたバッファを子がフラッシュしないよう `_exit` を使います。
- `WIFEXITED / WEXITSTATUS` と `WIFSIGNALED / WTERMSIG` で終了状態を解釈します。139 = 128 + 11 (SIGSEGV) は、シェルが `$?` で使うのと同じ慣例です。

⚠ 修正版

引き継いだ版（`redirection.c`）は `O_TRUNC` なしで出力を開いていたため（以前の長い実行の残りが残っていた）、また `write` の結果を無視していました。課題文では `lance` という名前でした。

探究：仮想メモリとプロセスの一生

課題。スライド 56～82 の内容を測る 4 つの短いプログラム：(1) `zones` はコード、データ、ヒープ、大きな `malloc`、`printf`、スタックのアドレスと、それぞれを含む `/proc/self/maps` の領域を表示します。何度か実行しましょう。(2) `lazy` は 1 GiB を予約し、その半分に触れる前後の常駐メモリとページフォルトを数えます。(3) `cow` は 256 MiB を埋めて fork し、子を読むときと書くときのフォルトを数えます。(4) `lifecycle` は `waitpid` の前後のゾンビと、引き取られた孤児を `ps` で見せます。

解答と実際の出力

C tds/td3/zones.c

```
/*
 * TD3 - exploration 1: the memory zones of a process (original slide 62,
 * updated).
 * Prints the address of the code, the data, the heap, a large malloc, a
 * library function and the stack,
 * and the region of /proc/self/maps that contains each one. Run it twice:
 * with ASLR (address space
 * layout randomization) the addresses change on every run, but the order of
 * the zones does not.
 */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int global_array[10];           /* uninitialized global: .bss */
static int initialized = 42;    /* initialized global: .data */

/* Finds the line of /proc/self/maps whose range contains addr and prints its
 * permissions and name. */
static void show(const char *what, const void *addr)
{
    FILE *maps = fopen("/proc/self/maps", "r");
    exit_if(maps == NULL, "/proc/self/maps");

    char line[512];
```

```

uintptr_t a = (uintptr_t)addr;
while (fgets(line, sizeof line, maps) != NULL) {
    unsigned long start, end;
    char perms[5], path[256] = "";
    if (sscanf(line, "%lx-%lx %4s %*s %*s %*s %255[^\n]", &start, &end,
perms, path) < 3)
        continue;
    if (a ≥ start && a < end) {
        char *name = path;
        while (*name == ' ')
            name++;
        printf("%-24s %#14lx  %s  %s\n", what, (unsigned long)a, perms,
*name ? name : "(anonymous)");
        break;
    }
}
exit_if(fclose(maps) == EOF, "fclose");
}

int main(void)
{
    int local = 0;
    void *small = malloc(10);
    void *big = malloc(64 << 20);          /* above M_MMAP_THRESHOLD (128 KiB):
glibc uses mmap */
    exit_if(small == NULL || big == NULL, "malloc");

    printf("%-24s %14s  %s  %s\n", "zone", "address", "perm", "region");
    show("code   (main)", (const void *)main);
    show("data   (initialized)", &initialized);
    show("bss    (global_array)", global_array);
    show("heap   (malloc 10 B)", small);
    show("heap   (end: sbrk(0))", (char *)sbrk(0) - 1);
    show("mmap   (malloc 64 MiB)", big);
    show("libc   (printf)", (const void *)printf);
    show("stack  (local)", &local);

    free(big);
    free(small);
    return EXIT_SUCCESS;
}

```

実際の出力

```
$ ./zones
zone                address    perm  region
code   (main)       0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data   (initialized) 0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss    (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap   (malloc 10 B)   0x5f938dc96010  rw-p  [heap]
heap   (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap   (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc   (printf)      0x71b345064080  r-xp  /usr/lib/x86_64-linux-
gnu/libc.so.6
stack  (local)        0x7fff6c1e68b4  rw-p  [stack]
```

実際の出力

```
$ for i in 1 2 3; do ./zones | grep main; done
code   (main)       0x57a1f784a350  r-xp  ~/fso/tds/bin/td3/zones
code   (main)       0x5d648e064350  r-xp  ~/fso/tds/bin/td3/zones
code   (main)       0x56e46bc21350  r-xp  ~/fso/tds/bin/td3/zones
```

領域の順序は変わりません（コード、データ、ヒープが下、ライブラリと `mmap` がその上、スタックが最上部）が、基点は実行のたびに変わります：**ASLR**。64 MiB の `malloc` はヒープにありません：glibc は 128 KiB を超えると `mmap` を使います。

C tds/td3/lazy.c

```
/*
 * TD3 - exploration 2: lazy allocation (original slides 59 and 61).
 * malloc of a large block returns at once and costs almost no physical
memory: the kernel only
 * reserves virtual addresses. Each page gets a physical frame the first time
it is touched, through a
 * (minor) page fault. The program measures the resident memory (VmRSS) and
the page faults
 * (getrusage) before and after touching half of the block.
 * ./lazy [MiB]          default: 1024 MiB
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <time.h>
#include <unistd.h>
```

```
#include "utils.h"

static long minor_faults(void)
{
    struct rusage ru;
    exit_if(getrusage(RUSAGE_SELF, &ru) == -1, "getrusage");
    return ru.ru_minflt;
}

/* Resident set size in KiB, from /proc/self/status. */
static long rss_kib(void)
{
    FILE *f = fopen("/proc/self/status", "r");
    exit_if(f == NULL, "/proc/self/status");
    char line[256];
    long kib = -1;
    while (fgets(line, sizeof line, f) != NULL)
        if (sscanf(line, "VmRSS: %ld kB", &kib) == 1)
            break;
    exit_if(fclose(f) == EOF, "fclose");
    return kib;
}

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void report(const char *step, long f0)
{
    printf("%-34s VmRSS = %8ld KiB    page faults so far = %ld\n", step,
    rss_kib(), minor_faults() - f0);
}

int main(int argc, char *argv[])
{
    size_t mib = argc > 1 ? (size_t)atol(argv[1]) : 1024;
    size_t size = mib << 20, page = (size_t)sysconf(_SC_PAGESIZE);
    long f0 = minor_faults();

    report("start", f0);
    char *p = malloc(size);
    exit_if(p == NULL, "malloc");
}
```

```
printf("malloc(%zu MiB) returned %p\n", mib, (void *)p);
report("after malloc", f0);

long before = minor_faults();
double t0 = now();
for (size_t i = 0; i < size / 2; i += page)    /* touch one byte of
every page of the first half */
    p[i] = 1;
double t1 = now();
long faults = minor_faults() - before;
report("after touching half of the pages", f0);
printf("pages touched: %zu (%zu bytes each)    page faults: %ld    %.0f ns
per fault\n",
        size / 2 / page, page, faults, faults ? (t1 - t0) * 1e9 / faults :
0.0);

free(p);
report("after free", f0);
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./lazy 1024
start                                VmRSS =      1756 KiB    page faults so far
= 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =      1952 KiB    page faults so far
= 8
after touching half of the pages      VmRSS =   526300 KiB    page faults so far
= 131081
pages touched: 131072 (4096 bytes each)    page faults: 131073    1968 ns per
fault
after free                            VmRSS =      2012 KiB    page faults so far
= 131081
```

`malloc(1 GiB)` は常駐メモリを変えません：アドレスを予約するだけです。初めて触れたページごとにページフォールトが起き、カーネルがフレームを割り当ててゼロで埋めます：フォールトの数はページ数と同じです。

C tds/td3/cow.c

```
/*
 * TD3 - exploration 3: copy-on-write after fork (original slide 74).
 * The parent fills a block, so all its pages are resident. After fork,
```

```

parent and child share the
    * physical pages, marked read-only. The child reads the whole block (no
copy: no page faults) and then
    * writes to a quarter of the pages: each first write causes a page fault in
which the kernel copies
    * that single page. The page faults are counted with getrusage.
    * ./cow [MiB]          default: 256 MiB
    */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static long minor_faults(void)
{
    struct rusage ru;
    exit_if(getrusage(RUSAGE_SELF, &ru) == -1, "getrusage");
    return ru.ru_minflt;
}

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

int main(int argc, char *argv[])
{
    size_t mib = argc > 1 ? (size_t)atol(argv[1]) : 256;
    size_t size = mib << 20, page = (size_t)sysconf(_SC_PAGESIZE), pages =
size / page;
    char *p = malloc(size);
    exit_if(p == NULL, "malloc");
    memset(p, 'x', size); /* every page is
now resident */
    printf("parent: %zu MiB filled (%zu pages of %zu bytes)\n", mib, pages,
page);
    fflush(stdout); /* do not
duplicate the stdio buffer */

```

```
double t0 = now();
pid_t pid = fork();
exit_if(pid == -1, "fork");
if (pid == 0) {
    printf("fork took %.2f ms (only the page tables are copied)\n",
(now() - t0) * 1e3);

    long f = minor_faults();
    unsigned long sum = 0;
    for (size_t i = 0; i < size; i += page)
        sum += (unsigned char)p[i];
    printf("child reads %zu pages: %ld page faults (shared, nothing
copied; sum %lu)\n", pages, minor_faults() - f, sum);

    f = minor_faults();
    double t1 = now();
    for (size_t i = 0; i < size / 4; i += page)
        p[i] = 'y';
    double t2 = now();
    long faults = minor_faults() - f;
    printf("child writes %zu pages: %ld page faults (one copy per page),
%.0f ns per copy\n",
        pages / 4, faults, faults ? (t2 - t1) * 1e9 / faults : 0.0);
    free(p);
    exit(EXIT_SUCCESS);
}
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
printf("parent: the child exited with status %d; my copy still says
'%c'\n", WEXITSTATUS(status), p[0]);
free(p);
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per
copy
parent: the child exited with status 0; my copy still says 'x'
```

`fork` の後、読むだけなら何もコピーされず（フォールト 0 回）、書くとき書いたページごとにちょうど 1 回のフォールト（1 回のコピー）が起きます。だから 256 MiB のプロセスの `fork` がミリ秒で済むのです：コピーするのはページテーブルだけです。

```
C tds/td3/lifecycle.c

/*
 * TD3 - exploration 4: the end of a process: zombies and orphans (original
 * slides 80-82 and 88).
 * 1. A child that exits stays as a ZOMBIE (state Z, <defunct>) until its
 * parent calls wait: the
 *     kernel keeps its PID and exit status. ps shows it before and after
 * waitpid.
 * 2. A process whose parent dies is an ORPHAN: the kernel gives it a new
 * parent, init (PID 1) or the
 *     nearest ancestor marked as "subreaper" (on a desktop, usually systemd
 * --user).
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static void pause_ms(long ms)
{
    struct timespec ts = { ms / 1000, (ms % 1000) * 1000000L };
    while (nanosleep(&ts, &ts) == -1)
        ;
}

static void ps(pid_t pid)
{
    char cmd[128];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,comm -p %d | sed 's/^/'", (int)pid);
    fflush(stdout);
    if (system(cmd) == -1)
        perror("system");
}

/* Name of a process, from /proc/PID/comm. */
static void comm_of(pid_t pid, char *name, size_t len)
```

```
{
    char path[64];
    snprintf(path, sizeof path, "/proc/%d/comm", (int)pid);
    FILE *f = fopen(path, "r");
    if (f == NULL || fgets(name, (int)len, f) == NULL)
        snprintf(name, len, "?");
    name[strcspn(name, "\n")] = '\0';
    if (f != NULL)
        fclose(f);
}

int main(void)
{
    /* 1. Zombie */
    pid_t child = fork();
    exit_if(child == -1, "fork");
    if (child == 0)
        _exit(7);
    pause_ms(200);
    printf("1) the child %d has exited, but nobody has called wait yet:\n",
(int)child);
    ps(child);
    int status;
    exit_if(waitpid(child, &status, 0) == -1, "waitpid");
    printf("    waitpid → WIFEXITED = %d, WEXITSTATUS = %d; now ps finds
nothing:\n", WIFEXITED(status), WEXITSTATUS(status));
    ps(child);

    /* 2. Orphan: the middle process exits, the grandchild is adopted */
    printf("2) orphan: my PID is %d\n", (int)getpid());
    fflush(stdout);
    pid_t middle = fork();
    exit_if(middle == -1, "fork");
    if (middle == 0) {
        pid_t grandchild = fork();
        exit_if(grandchild == -1, "fork");
        if (grandchild == 0) {
            pid_t before = getppid();
            pause_ms(200);
            /* meanwhile its parent
exits */
            pid_t after = getppid();
            char name[64];
            comm_of(after, name, sizeof name);
            printf("    grandchild %d: parent was %d, now it is %d (%s)\n",
(int)getpid(), (int)before, (int)after, name);
```

```

        exit(EXIT_SUCCESS);
    }
    exit(EXIT_SUCCESS);                /* the grandchild becomes
an orphan */
    }
    exit_if(waitpid(middle, NULL, 0) == -1, "waitpid");
    pause_ms(400);                      /* let the grandchild
report */
    return EXIT_SUCCESS;
}

```

実際の出力

```

$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
grandchild 356679: parent was 9771, now it is 9771 (systemd)

```

終了した子は `waitpid` がコード (7) を回収するまで状態 `Z` です。孤児になった孫は `init` ではなく、ユーザーセッションの `systemd` (`subreaper`、`prctl(PR_SET_CHILD_SUBREAPER)`) に引き取られます。

アニメーションで見る → A4.1 仮想メモリ・A4.2 コピーオンライト・A4.3 状態

チェックリスト

生成AIチャレンジ・ミニシェル



目標。 `launch` を最小限のコマンドインタプリタにしましょう：行を読み、`fork`/`execvp` でコマンドを実行し、`> file` に対応し、終了状態を表示します。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

私の `launch.c` [貼り付け] をもとに、次のような C のミニシェルを書くのを手伝ってください：`getline` で行を読み、単語に分け、`fork + execvp` でコマンドを実行し、`dup2` で `> file` のリダイレクトに対応し、`cd` と `exit` を組み込みコマンドとして実装し（`cd` が外部プログラムにできない理由を説明して）、コマンドが終わったら `[status N]` か `[signal SIGxxx]` を表示する。あなたの答えを見せる前に、各部分を私に書かせてください。

結果の確かめ方

- `ls -l > out.txt` でファイルが作られ、シェルは動き続ける。
- `cd /tmp` の後の `pwd` が `/tmp` と表示する。
- `./sefault` を実行するとシグナルが表示され、ゾンビプロセスは残らない（`ps`）。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。